

Q1:

Coding against an interface is better because it makes the code more flexible and easier to maintain.

You can change the concrete class or use another implementation without changing the code that depends on it.

Q2:

when the classes share common behavior or data, and you want to provide some implementation that they can reuse.

Q3:

it tells C# how to compare objects when sorting them.

Q4:

to create a new object by copying the values of an existing object of the same class.

Q5:

helps resolve naming conflicts when two interfaces have methods with the same name.

It allows the class to provide a separate implementation for each interface, so there is no conflict between them.

Q6:

both structs and classes support encapsulation,

but classes are generally more flexible because they can use inheritance and reference-type behavior,

while structs are value types and have more limitations.

Q7:

Abstraction means hiding unnecessary implementation details and showing only the essential features.

It is related to encapsulation because both hide details,

but abstraction focuses on what an object does, while encapsulation focuses on how its data and implementation are protected.

last in part01:

by allowing objects to be created in different ways, depending on the data available. This makes the class more flexible and easier to use.

part02:

2- Coding against an interface means writing your code to depend on an interface (abstraction) instead of a specific class this makes the code more flexible because you can change or replace the implementation without changing the code that uses it.

Code against abstraction, not concreteness means depending on what the object can do, not on how it does it.

3- Abstraction as a guideline means we should hide unnecessary implementation details and expose only what is important to the user. We can implement this using interfaces and abstract classes, where we define what should be done without forcing the user to know the internal implementation.



Salma Mahmoud · You

Computer Science Student at October 6 University | A...
2h · 🌐

مش كل Class بنعمله هدفه إننا نعمل منه Object.

ده كان من الحاجات اللي خلتني أفهم الـ Abstract Class بشكل مختلف.

لما بدأت أتعلم OOP في C#, فكرة إن فيه Class مينفعش تعمل منه Object مباشرة كانت غريبة بالنسبالي شوية. لكن لما فكرت فيها ببساطة، لقيت إن الـ Abstract Class مش معمول علشان تستخدمه لوحده، هو معمول علشان يكون Base Class تشارك من خلاله الحاجات المشتركة، وتسبب الي بيختلف للـ Classes اللي هتورث منه.

مثلاً لو عندنا:

```
public abstract class Shape
{
    public string Name { get; set; }
    public void DisplayName() =>
        Console.WriteLine(Name);
    public abstract double CalculateArea();
}

public class Circle : Shape
{
    public override double CalculateArea() => Math.PI * 5 * 5;
}

public class Rectangle : Shape
{
    public override double CalculateArea() => 10 * 5;
}
```

هنا Shape بيحدد حاجات عامة لأي Shape، لكن مش هو اللي بيحدد كل شكل كـ Class.



Leave your thoughts here...



Home



My Network



Post



Notifications



Jobs



هنا Shape يحدد حاجات عامة لأي Shape، لكن مش هو اللي بيحدد مساحة كل شكل. كل Class هيحسبها بطريقته.

والنقطة المهمة إن الـ Abstract Class بتخليك تقول:

"أنا عارف إن كل Shape لازم يكون عنده CalculateArea، بس طريقة الحساب مش واحدة."

وفي نفس الوقت تقدر تحط جواها Methods و Properties عادية، مش لازم كل حاجة تبقى abstract.

يعني باختصار:

Derived Classes لازم الـ Rules + حاجات مشتركة = Abstract Class. تلتزم بيها.

وأعتقد إن أحسن طريقة نفهم بيها الـ OOP هي إننا نسأل نفسنا مش بس "الـ Concept ده بيعمل إيه؟"، لكن كمان "ليه محتاجينه من الأساس؟".

ودي كانت واحدة من الحاجات اللي خلت فكرة الـ Abstract Class تبقى أوضح بالنسبالي.

#CSharp #OOP #DotNet #Programming

Show translation



Abstract Class
in C#

```
public abstract class Shape
{
    public string Name { get; set; }
}
```

Shape (abstract)
+ Name : string

Not every class is meant to be instantiated.

Defines what's common + the rules for the derived



Leave your thoughts here...



Home



My Network



Post



Notifications



Jobs

5:39



بس ان Concept ده بيعمل إيه؟ / نحن نمان فيه محتاجينه من الأساس؟

ودي كانت واحدة من الحاجات اللي خلت فكرة الـ Abstract Class تبقى أوضح بالنسبالي.

#CSharp #OOP #DotNet #Programming

Show translation



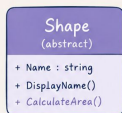
Abstract Class in C#

Not every class is meant to be instantiated.

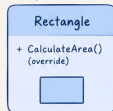
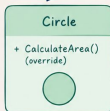
```
public abstract class Shape
{
    public string Name { get; set; }
    public void DisplayName() =>
        Console.WriteLine(Name);
    public abstract double CalculateArea();
}

public class Circle : Shape
{
    public override double CalculateArea() =>
        Math.PI * 5 * 5;
}

public class Rectangle : Shape
{
    public override double CalculateArea() =>
        10 * 5;
}
```



Defines what's common + the rules for the derived classes.



Abstract Class =
shared members +
rules for the derived classes.

#CSharp #OOP #DotNet #Programming

Yousef Mohamed and 2 others



Leave your thoughts here...



Home



My Network



Post



Notifications



Jobs